Introduction au tri fusion: quelques opérations sur les listes chaînées

Quelques fonctions ou portions de code utiles

Travail

Prendre connaissance et s'assurer de la bonne compréhension des classes ou fonctions proposées ci-après. Exécuter les cellules.

```
class Cell:
    """ Classe cellule pour liste chainée"""

    def __init__(self, v, s):
        self.val = v
        self.suiv = s
```

```
# TESTS a voir éventuellement sur pythontutor
lst1 = Cell(10, Cell(5, Cell(-4, Cell(0, Cell(1, None)))))
lst2 = Cell(2, lst1)
```

```
def taille(l):
    return 1 if l.suiv is None else 1 + taille(l.suiv)
```

```
taille(lst2)
```

```
6
```

```
def affiche_list(l):
    while l is not None:
        print(l.val, ' --> ', end='')
        l = l.suiv
    print('None')
```

```
def copie(l):
    """
    renvoie une liste chaînée, copie de l
    """
    return None if l is None else Cell(l.val, copie(l.suiv))
```

```
affiche_list(lst2)
```

```
2  --> 10  --> 5  --> -4  --> 0  --> 1  --> None
```

Couper une liste en deux

Il s'agit de séparer les éléments d'une liste en deux listes de même taille (*à un élément près si la taille de la liste originale est impaire*).

Proposition 1

Travail

- trouver la taille n de la liste;
- mettre les $\frac{n}{2}$ premiers dans la première liste et les autres dans la seconde liste.

Compléter la fonction `coupe_v1`.

```
def coupe_v1(l):
    """
    coupe l en deux parties (égales si la longueur de l est paire), chacune
    constituant une nouvelle liste.
    Renvoie deux listes chainées l1 et l2
    """

    l1, l2 = None, None
    n = taille(l) # Taille de la liste
    cpt = 0 # Un compteur
    # A compléter
    ...
    return l1, l2
```

```
# TESTS
l1, l2 = coupe_v1(lst2)
affiche_list(lst2)
affiche_list(l1)
affiche_list(l2)
```

```
2  --> 10  --> 5  --> -4  --> 0  --> 1  --> None
5  --> 10  --> 2  --> None
1  --> 0  --> -4  --> None
```

Proposition 2

Travail

- prendre deux éléments de la liste (si possible);
- disposer un élément dans chaque liste `l1` et `l2`.

Compléter la fonction `coupe_v2`.

```
def coupe_v2(l):
    """
    coupe l en deux parties (égales si la longueur de l est paire).
    renvoie deux listes chainées l1 et l2
    """

    l1, l2 = None, None
    while l is not None:
        # A compléter
```

```
# TESTS
l1, l2 = coupe_v2(lst1)
affiche_list(lst1)
affiche_list(l1)
affiche_list(l2)
```

Proposition 3

Travail (moins facile)

On parcours la liste originale et on place la valeur une fois dans `l1`, une fois dans `l2`.
Écrire une fonction `coupe_v3` qui répond à cette consigne.

```
def coupe_v3(l):
    pass
```

Fusionner deux listes

Travail

On considère deux listes chaînées `l1` et `l2` que l'**on suppose triées par ordre croissant**. On demande d'écrire une fonction `fusion` qui prend en paramètre `l1` et `l2` et qui renvoie une liste chaînée contenant tous les éléments de `l1` et `l2` triés par ordre croissant. La récursivité est bienvenue !

```python
def fusion(l1, l2):
    """
    renvoie la fusion ordonée de deux listes triées;
    l1, l2: listes chaînées
    """
    # Les cas de base
    if l1 is None: return l2
    if l2 is None: return l1
    # Cas récursif
    # A compléter
```

```python
# TESTS
x = Cell(3, Cell(8, Cell(13, Cell(17, None))))
y = Cell(-2, Cell(1, Cell(7, Cell(11, None))))
affiche_list(x)
affiche_list(y)
liste_finale = fusion(x, y)
affiche_list(liste_finale)
```

```
3  --> 8  --> 13  --> 17  --> None
-2  --> 1  --> 7  --> 11  --> None
-2  --> 1  --> 3  --> 7  --> 8  --> 11  --> 13  --> 17  --> None
```